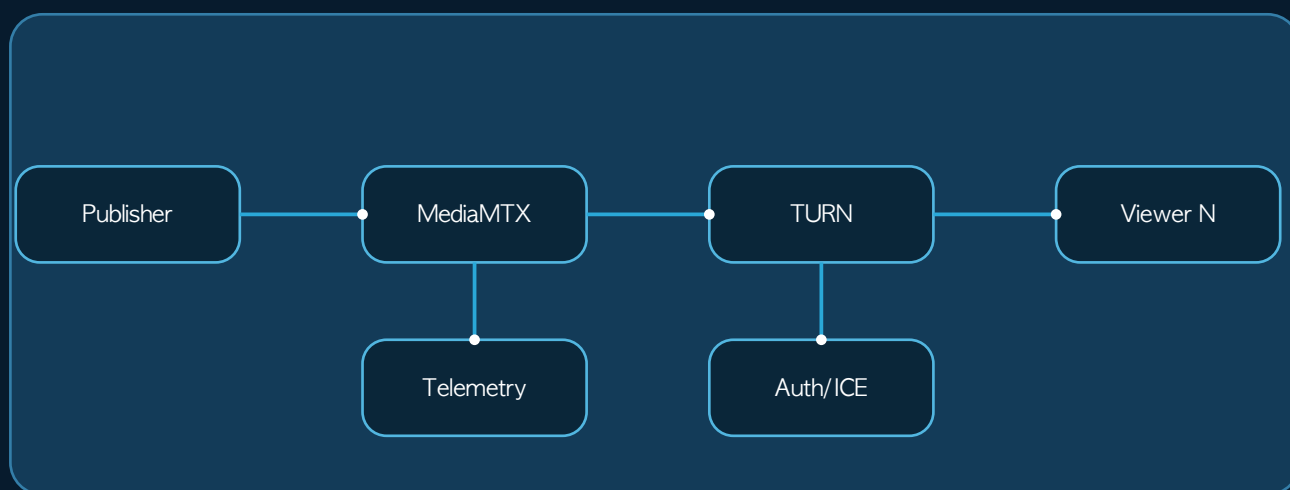


GCS-Saker

TURN 서버 부하 절감 최적화 보고서

v0.3 / N:M 구조, STUN/TURN signaling 내부 로직, relay port 고갈 모델 반영



핵심 결론

파이프라인 CPU 과부하보다 N:M viewer fan-out, TURN allocation, relay port range, fd/conntrack, NIC 대역폭이 먼저 병목이 될 가능성이 높다. 따라서 우선순위는 direct ICE 성공률, 후보 정리, lazy player, port/OS 한계 관리, credential/quota 순서다.

고려한 요소

이번 v0.3에서 N:M 구조, signaling 내부 로직, TURN 자원 고갈 가능성을 명시적으로 반영

이 문서는 단순히 TURN 서버를 빠르게 만드는 방법만 다루지 않는다. GCS-Saker가 실제 운영에서 만날 수 있는 viewer 증가, 스트림 타일 증가, 공유기/방화벽 포워딩, 폐쇄망, 자체 STUN/TURN, 서버 2대 역할 분리까지 같이 본다.

용량/성능 관점

- N:M 구조: publisher P개와 viewer V명이 있고, viewer가 여러 stream tile을 동시에 열 수 있다.
- TURN allocation: relayed peer connection 수에 비례해 relay endpoint, permission, channel, fd가 증가한다.
- relay port range: PoC의 49160~49200 같은 좁은 범위는 작은 실험용으로는 충분하지만 운영 N:M에는 부족할 수 있다.
- pipeline 과부하: MediaMTX가 passthrough 중심이면 transcoding CPU보다 relay/NIC/port 한계가 먼저 올 가능성이 높다.

운영/보안 관점

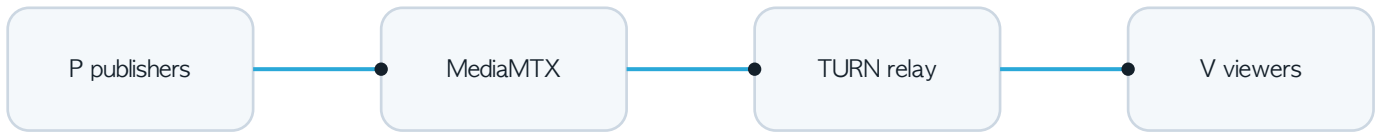
- NAT/방화벽: UDP 차단, NAT hairpin 실패, 잘못된 advertised candidate가 direct ICE를 막는다.
- NIC/conntrack/fd: TURN은 packet relay 서버라 OS 네트워크 한계가 application 코드보다 먼저 드러날 수 있다.
- 보안: TURN credential TTL, quota, group 권한이 없으면 무단 relay가 성능 문제로 이어진다.
- 폐쇄망: 내부망에서는 private candidate가 오히려 가장 빠른 경로이며 공개망 profile과 분리해야 한다.

결론

TURN 서버 부담의 본질은 CPU pipeline 하나가 아니라 '얼마나 많은 peer connection이 relay로 빠지는가'와 '그 relay가 포트, 파일 디스크립터, 커널 네트워크 큐, NIC 대역폭을 얼마나 소비하는가'다. 따라서 direct ICE 성공률과 viewer-side lazy 연결을 먼저 잡는 것이 가장 안전하다.

N:M 용량 모델

TURN port 부족과 allocation 폭증을 먼저 추정해야 함



간단 추정식

- 동시 viewer connection $\sim V \times \text{선택/실시간 타일 수 } S$
- relay peer connection $\sim (\text{publisher relay 수} + \text{viewer connection 수}) \times \text{relay 비율 } R$
- TURN relay 대역폭 $\sim \sum (\text{선택 스트림 bitrate} \times \text{relay viewer 수})$
- relay port 압박 $\sim \text{allocation 수} \times \text{ICE component 수} \times \text{transport 정책} + \text{stale allocation 여유}$

현재 PoC port range 주의

49160~49200은 41개 포트 범위다.
BUNDLE/RTCP mux 환경에서는 과거 RTP/RTCP 분리보다 효율적일 수 있지만, N:M 운영에서 allocation이 늘면 이 범위는 빠르게 부족해질 수 있다.

pipeline 과부하 판단

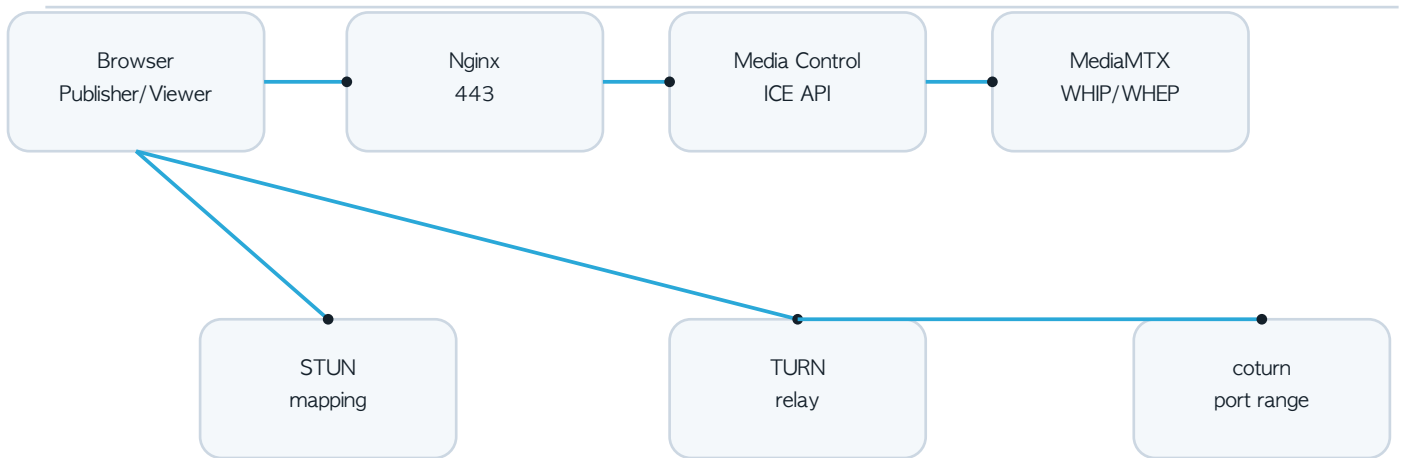
MediaMTX가 passthrough 중심이면 encoding pipeline CPU보다 TURN relay의 NIC, fd, conntrack, port range가 먼저 병목일 수 있다. transcoding을 넣는 순간 CPU 병목 모델을 따로 봐야 한다.

운영 기준

운영 전에는 stream 1개 기준이 아니라 P, V, S, R을 바꿔가며 allocation 수와 relay Mbps를 계측해야 한다. 포트 고갈은 평균 latency보다 먼저 '연결 실패/간헐적 ICE 실패'로 나타날 수 있으므로 연결 성공률과 coturn allocation 로그를 같이 봐야 한다.

Signaling 전체 흐름

WHEP/WHIP, STUN, TURN이 서로 만나는 지점



단계별 흐름

- 1. 사용자가 로그인하고 stream 권한을 확인한다. 이 단계는 JWT/refresh/session 정책과 group 권한을 탄다.
- 2. 프론트가 `/media-control/api/v1/streams/ice-servers`에서 STUN/TURN 서버 목록과 짧은 TTL credential을 받는다.
- 3. 브라우저는 RTCPeerConnection을 만들고 ICE gathering을 시작한다. host, srflx(STUN), relay(TURN) candidate가 생성된다.
- 4. 송출자는 WHIP offer를 MediaMTX에 POST하고, 수신자는 WHEP offer를 MediaMTX에 POST한다. MediaMTX는 SDP answer를 돌려준다.
- 5. 브라우저와 MediaMTX는 ICE connectivity check를 수행하고 selected candidate pair를 고른다.
- 6. direct가 실패하고 relay가 선택되면 SRTP/RTCP packet이 coturn relay port를 통해 흐른다.
- 7. dashboard는 getStats, backend event log, coturn allocation log를 합쳐 direct/relay, RTT, jitter, packet loss를 기록한다.

STUN 내부 로직

STUN은 media relay가 아니라 NAT mapping 발견용

무슨 일이 일어나나

- 브라우저가 STUN server로 Binding Request를 보낸다. 이 요청은 media packet이 아니라 NAT mapping 확인용 control packet이다.
- STUN server는 응답에 XOR-MAPPED-ADDRESS를 담아 돌려준다. 브라우저는 이 주소를 server-reflexive(srflx) candidate로 등록한다.
- srflx candidate는 외부에서 이 클라이언트를 어떻게 볼 수 있는지 알려준다. 이 candidate로 ICE check가 성공하면 TURN relay가 필요 없다.
- STUN은 packet을 계속 중계하지 않는다. 연결 확인과 consent freshness용 keepalive는 있지만 영상/음성 대역폭을 먹는 relay가 아니다.

건드리는 곳

- Frontend: RTCPeerConnection
iceServers
- Backend: ICE server DTO, credential
TTL
- Nginx: ICE API route
- Firewall: STUN/TURN port
outbound/inbound
- Profile: public/closed/hybrid

장애 신호

- srflx candidate가 없음
- ICE checking에서 relay로만 성공
- 같은 망인데도 TURN 사용률이 높음
- candidate에 잘못된 private/Docker IP 노출
- NAT hairpin 실패

최적화 포인트

STUN을 자체 서버로 바꾸는 것은 폐쇄망 독립성에는 좋지만, 공개망에서 Google STUN보다 항상 빠르다고 보장할 수는 없다. 핵심은 STUN 서버 자체보다 '실제 도달 가능한 candidate만 주고받는가', '공개망/폐쇄망 profile이 섞이지 않는가', 'direct 실패 원인을 로그로 남기는가'다.

TURN 내부 로직

relay port, allocation, permission이 부하의 실체

TURN allocation 순서

- 1. 브라우저가 TURN Allocate Request를 보낸다. long-term credential 방식이면 nonce/realm 검증이 포함된다.
- 2. coturn은 인증이 맞으면 relay endpoint를 할당한다. 이때 `min-port/max-port` 범위 안의 relay port가 소비된다.
- 3. 브라우저는 이 relay 주소를 relay candidate로 ICE에 등록한다.
- 4. 실제 상대와 통신하려면 CreatePermission으로 peer IP를 허용한다. 더 효율적인 전송을 위해 ChannelBind가 붙을 수 있다.
- 5. media packet은 브라우저 -> TURN -> MediaMTX 또는 MediaMTX -> TURN -> 브라우저로 흐른다. 이 구간이 TURN 대역폭과 NIC를 직접 소비한다.
- 6. 세션 중 Refresh가 allocation 수명을 갱신하고, 종료/timeout 시 relay port가 반환된다.

부하가 생기는 지점

- relay port range
- allocation table
- permission/channel table
- file descriptor
- UDP socket buffer
- conntrack/NAT table
- NIC packet rate와 Mbps

설정 touch point

- TURN realm/shared secret
- credential TTL
- external-ip
- min-port/max-port
- total-quota/user-quota
- max-bps
- UDP/TCP/TLS listener

왜 port 부족이 문제인가

TURN은 단순 API 서버가 아니라 relay endpoint를 실제로 할당한다. 현재 PoC처럼 49160~49200 범위가 작으면 소수 테스트는 통과해도 N:M viewer가 늘 때 allocation 실패, 간헐적 ICE 실패, 재연결 루프가 먼저 나타날 수 있다.

BUNDLE/RTCP mux 덕분에 과거보다 port 소비가 줄 수는 있지만, 운영 용량은 반드시 실제 allocation 로그로 잡아야 한다.

WHEP/WHIP와 장애 지점

signaling은 성공했는데 media가 안 오는 경우를 나누어 본다

송출 WHIP

- Publisher가 local media track을 만들고 RTCPeerConnection offer를 생성한다.
- WHIP endpoint로 SDP offer를 POST한다. Nginx는 `/webrtc/.../whip`를 MediaMTX로 proxy한다.
- MediaMTX가 SDP answer를 반환하면 브라우저는 setRemoteDescription을 수행한다.
- ICE가 connected/completed가 되면 SRTP media가 흐르고 stream registry에 active 상태가 반영된다.

수신 WHEP

- Dashboard가 stream 목록에서 선택한 stream path로 WHEP offer를 POST한다.
- MediaMTX answer를 받은 뒤 remote track event가 발생해야 video/audio element에 media stream이 붙는다.
- signaling 성공 + ICE connected인데 화면이 안 나오면 track binding, autoplay/muted 정책, codec mismatch, selected candidate media path를 확인한다.
- GET에서 405가 나는 것은 path 도달 확인일 수 있지만, 실제 WHEP 검증은 POST offer와 SDP answer로 봐야 한다.

오류를 어디서 볼 것인가

- 401: dashboard/auth 권한 또는 TURN credential 만료
- 404: stream path 없음, registry와 MediaMTX path 불일치
- 405: 잘못된 method. WHEP/WHIP는 실제로 POST offer가 필요
- 502: Nginx -> MediaMTX proxy, container health, upstream port 문제
- ICE connected but no media: candidate는 붙었지만 track/codec/relay port/media flow 문제

01. Direct ICE 성공률 우선

우선순위: 최상 / 난이도: 중

TURN을 빠르게 만드는 것보다 TURN을 타지 않게 만드는 것이 가장 큰 절감책이다.

고려 요소

- NAT 유형
- candidate 품질
- 방화벽
- 동일 대역 direct 가능성

예상 개선

망 환경에 따라 relay 트래픽 20~80% 감소 가능. 동일 현장망/폐쇄망에서는 가장 큰 절감 폭을 기대할 수 있다.

근거

WebRTC ICE는 host/srflx/relay 후보를 수집한 뒤 실제 연결 가능한 candidate pair를 선택한다. host 또는 srflx가 성공하면 media packet은 TURN relay를 지나지 않는다.

적용 방안

- 기본 정책은 iceTransportPolicy=all로 유지하고 relay 강제는 진단 모드로만 둔다.
- getStats로 selected candidate type, ICE connected latency, first-frame latency를 기록한다.
- direct 실패 원인을 private candidate, advertised host, UDP 차단, NAT hairpin 실패로 분류한다.

검증 지표

- relay 비율
- ICE connected latency
- first-frame latency
- packet loss

02. MediaMTX candidate 정리

우선순위: 최상 / 난이도: 중

외부 클라이언트가 닿을 수 없는 Docker/private 후보를 줄여 불필요한 TURN fallback을 막는다.

고려 요소

- SDP candidate
- 공개망/폐쇄망 profile
- NAT address
- WHEP 응답

예상 개선

TURN fallback 10~50% 감소 가능. coturn 403 Forbidden IP 노이즈도 줄어든다.

근거

SDP에 loopback/private/Docker 주소가 섞이면 외부 단말은 실패 후보를 먼저 시도하고 relay로 빠질 가능성이 커진다.

적용 방안

- public profile에는 외부 도달 가능한 host만 advertised 한다.
- closed profile에는 내부 도달 가능한 host와 자체 STUN/TURN을 쓴다.
- 후보 요약 로그를 남겨 private/loopback 후보 수를 회귀 테스트한다.

검증 지표

- candidate type 분포
- private 후보 수
- relay fallback 수
- coturn 403 로그

03. Dashboard WebRTC lazy 연결

우선순위: 최상 / 난이도: 중상

작게 보이는 모든 타일을 동시에 WebRTC로 붙이지 말고 선택/확대 타일 위주로 연결한다.

고려 요소

- N:M viewer 증가
- tile fan-out
- 브라우저 decoder
- TURN allocation

예상 개선

동시 TURN allocation 30~70% 감소 가능. 스트림 수와 viewer 수가 늘수록 효과가 커진다.

근거

N:M 구조에서 대시보드 viewer가 4개 타일을 동시에 열면 viewer 1명당 최대 4개의 WHEP peer connection이 생길 수 있다.

적용 방안

- 선택/확대/핀/팝업된 스트림만 실시간 WebRTC를 유지한다.
- 비선택 타일은 snapshot, paused preview, low-rate HLS preview로 둔다.
- 탭 비활성 또는 viewport 밖 카드의 peer connection을 정리한다.

검증 지표

- active player 수
- allocation 수
- dashboard CPU
- memory
- reconnect latency

04. 로봇/드론 native ingest

우선순위: 상 / 난이도: 상

장비가 이미 지원하는 RTSP/RTP/SRT/WHIP를 우선 사용해 송출 측 WebRTC relay 부담을 줄인다.

고려 요소

- 장비 protocol
- codec passthrough
- telemetry 분리
- ingest 안정성

예상 개선

송출 측 TURN 사용 최대 100% 제거 가능.
transcoding 회피 시 서버 CPU 20~60% 절감 가능.

근거

로봇/드론/IP 카메라는 WebRTC보다 RTSP/RTP/SRT를 native로 지원하는 경우가 많다. 서버가 재인코딩하지 않으면 CPU와 지연이 줄어든다.

적용 방안

- 장비별 ingest matrix를 만든다: RTSP, RTP, SRT, WHIP, vendor SDK.
- 영상은 MediaMTX ingest, telemetry는 MQTT/HTTP/MAVLink 계열로 분리한다.
- timestamp를 기준으로 video/audio/telemetry를 dashboard에서 다시 결합한다.

검증 지표

- publisher TURN 사용률
- server CPU
- ingest-to-play latency
- codec passthrough

05. UDP relay 우선, TCP/TLS fallback

우선순위: 상 / 난이도: 하중

실시간 미디어에는 UDP TURN을 우선 사용하고 TCP/TLS TURN은 막힌 망의 마지막 fallback으로 둔다.

고려 요소

- head-of-line blocking
- 무선 packet loss
- 방화벽 정책
- fallback 연결성

예상 개선

relay RTT와 jitter 5~30% 개선 가능. packet loss가 있는 환경일수록 차이가 커질 수 있다.

근거

TCP relay는 손실 시 뒤 packet까지 대기하는 특성이 있어 실시간 영상/음성 지연에 불리하다.

적용 방안

- ICE server list 순서를 UDP TURN 우선으로 둔다.
- TCP/TLS TURN은 UDP 차단망용 fallback으로 남긴다.
- selected candidate protocol을 통계로 남겨 정책 효과를 확인한다.

검증 지표

- selected protocol
- RTT
- jitter
- packet loss
- relay Mbps

06. coturn quota와 bandwidth guardrail

우선순위: 상 / 난이도: 중

속도 향상보다는 폭주와 오남용으로 TURN 전체가 죽는 상황을 막는 안전장치다.

고려 요소

- 무단 relay
- quota
- bandwidth cap
- 장애 격리

예상 개선

평상시 평균 지연 개선은 작지만 장애 상황에서 NIC/CPU 포화를 방지한다.

근거

TURN은 public relay가 되기 쉽다. 사용자별/전체 quota가 없으면 비정상 단말 하나가 allocation과 대역폭을 잠식할 수 있다.

적용 방안

- total-quota, user-quota, max-bps 후보값을 부하 테스트로 산정한다.
- publisher, viewer, admin 계정의 TURN 권한을 분리한다.
- quota 초과는 custom error와 event log로 사용자에게 명확히 보여준다.

검증 지표

- quota 초과 이벤트
- allocation 거절 수
- relay Mbps
- CPU/NIC

07. TURN primary/secondary 분산

우선순위: 상 / 난이도: 중

3478/3479를 단순 백업이 아니라 부하 분산과 장애 격리 용도로 사용한다.

고려 요소

- 두 서버 역할
- ICE list 순서
- relay port range
- failover

예상 개선

단일 TURN 서버 부하 30~50% 분산 가능. 실제 효과는 ICE list와 client 분배 정책에 좌우된다.

근거

TURN relay는 packet이 서버 NIC를 직접 통과한다. 인스턴스를 분산하면 단일 NIC/CPU 병목과 장애 반경이 줄어든다.

적용 방안

- viewer/publisher/group 단위로 primary TURN 우선순위를 다르게 준다.
- 두 TURN 서버의 relay port range를 겹치지 않게 분리한다.
- health 기반 ICE API가 비정상 TURN을 자동 제외한다.

검증 지표

- TURN별 allocation
- TURN별 relay Mbps
- TURN별 CPU/NIC
- failover 성공률

08. 폐쇄망 direct profile

우선순위: 상 / 난이도: 중상

폐쇄망에서는 내부 IP 후보가 가장 빠른 경로일 수 있으므로 공개망과 다른 profile을 사용한다.

고려 요소

- 인터넷 단절
- 내부 DNS
- 자체 STUN/TURN
- offline map/time

예상 개선

폐쇄망 relay 사용 50~90% 감소 가능. 내부 LAN 품질이 좋으면 거의 direct로 처리할 수 있다.

근거

공개망에서는 private candidate가 쓸모없지만, 한 대역 폐쇄망에서는 private candidate가 실제 최저 지연 경로다.

적용 방안

- public, closed, hybrid profile을 분리한다.
- closed profile에는 자체 STUN/TURN/time/map endpoint를 넣는다.
- 인터넷 차단 smoke test를 정기적으로 돌린다.

검증 지표

- closed relay 비율
- offline smoke 성공률
- internal direct latency
- 외부 의존 수

09. Adaptive bitrate / resolution / fps

우선순위: 중상 / 난이도: 상

relay 경로 또는 작은 tile에는 bitrate와 fps를 낮춰 relay Mbps를 직접 줄인다.

고려 요소

- relay Mbps
- operator 식별성
- tile 크기
- packet loss

예상 개선

relay Mbps 20~60% 감소 가능. 임무별 최소 식별 품질을 지켜야 한다.

근거

TURN relay 트래픽은 bitrate에 거의 선형으로 증가한다. 비선택 타일 품질을 낮추면 서버 대역폭과 client decode 부하가 같이 줄어든다.

적용 방안

- selected stream은 고품질, overview tile은 저품질로 둔다.
- relay candidate 감지 시 downshift 정책을 적용한다.
- AI event/thumbnail/telemetry를 영상 품질 보조 정보로 활용한다.

검증 지표

- bitrate
- fps
- frame drop
- packet loss
- 식별 가능성

10. coturn OS/NIC 튜닝

우선순위: 중상 / 난이도: 중상

TURN 구현체를 바꾸기 전에 fd, UDP buffer, conntrack, NIC drop 같은 운영 한계를 먼저 제거한다.

고려 요소

- file descriptor
- UDP buffer
- conntrack
- NIC queue
- softirq

예상 개선

동시 세션 한계 2~5배 개선 가능성. 서버 스펙과 커널 설정에 크게 의존한다.

근거

relay 서버는 application CPU보다 커널 네트워크 큐와 fd 제한에 먼저 막힐 수 있다.

적용 방안

- systemd LimitNOFILE, ulimit, UDP rmem/wmem, conntrack limit을 점검한다.
- NIC drop, softirq, receive errors를 지표로 잡는다.
- 튜닝은 서버 2에서 먼저 검증 후 서버 1에 반영한다.

검증 지표

- open files
- UDP receive errors
- conntrack
- NIC drops
- relay Mbps

11. Ephemeral TURN credential

우선순위: 중 / 난이도: 중

짧은 TTL credential로 허가된 세션만 relay를 쓰게 해서 보안과 자원 보호를 동시에 잡는다.

고려 요소

- credential TTL
- 권한 모델
- 토큰 갱신
- 무단 relay

예상 개선

정상 트래픽 속도 개선은 작지만 무권한 relay 사용을 크게 줄인다.

근거

TURN credential이 오래 살아 있거나 공유되면 외부에서 relay를 남용할 수 있다.

적용 방안

- media-control이 stream 권한을 확인한 뒤 짧은 TTL TURN credential을 발급한다.
- refresh token과 TURN credential TTL을 분리한다.
- 만료와 재발급 실패를 UI 상태와 event log에 남긴다.

검증 지표

- credential 발급 수
- 무권한 allocation
- TTL
- refresh 실패율

12. Rust TURN PoC는 후순위

우선순위: 후순위 / 난이도: 상

Rust 전환은 coturn이 실제 CPU 병목으로 확인된 뒤 shadow benchmark로만 시작한다.

고려 요소

- 운영 성숙도
- 보안 업데이트
- 브라우저 호환
- relay 본질 비용

예상 개선

CPU 0~30% 개선 가능성. 대신 운영 리스크가 증가할 수 있다.

근거

TURN 부담의 대부분은 언어보다 relay traffic, port, NIC, quota, fd에서 온다. 구현체 교체는 마지막 카드다.

적용 방안

- coturn과 Rust TURN을 같은 세션/bitrate/port range로 비교한다.
- UDP/TCP/TLS, auth, quota, log, metrics 호환성을 확인한다.
- 운영 전환이 아니라 shadow benchmark와 PoC issue로만 진행한다.

검증 지표

- CPU
- memory
- allocation latency
- relay RTT
- browser compatibility

검증 계획

평균값보다 연결 실패, port 고갈, tail latency를 먼저 봐야 함

1. N:M 부하 시나리오

- P=1/4/8 publisher, V=1/4/10 viewer, S=1/4 active tile 조합으로 반복한다.
- R=direct, mixed, relay-only 세 profile로 나눠 first-frame latency와 relay Mbps를 기록한다.
- port range를 PoC 범위와 확장 범위로 바꿔 allocation 실패가 언제 시작되는지 확인한다.

2. 장애/보안 시나리오

- TURN primary down, TURN secondary only, MediaMTX restart, Redis restart에서 degraded behavior를 고정한다.
- 만료 credential, 잘못된 group 권한, quota 초과, UDP 차단망에서 사용자 메시지와 event log를 검증한다.
- closed profile은 인터넷 차단 상태에서 자체 STUN/TURN/time/map만으로 smoke test를 수행한다.

3. 수집해야 하는 숫자

- ICE connected latency / first-frame latency / audio-video sync offset
- selected candidate type과 protocol / direct:relay 비율
- coturn allocation 수 / permission 수 / relay Mbps / denied allocation 수
- fd 사용량 / conntrack 사용량 / NIC drop / UDP receive errors / CPU softirq

권장 적용 순서

서버 구조를 크게 뒤집기 전에 효과가 큰 것부터

1단계

Candidate 정리 + direct ICE 계측 + lazy player

릴레이 비율을 먼저 낮춘다. 가장 큰 절감 가능성이 있고 UI/UX에도 직접 도움된다.

2단계

TURN port range/OS limit 점검 + quota

PoC port 범위는 운영 N:M에 부족할 수 있다. 포트, fd, conntrack, NIC를 먼저 계측한다.

3단계

Primary/secondary TURN 분산 + ephemeral credential

부하와 장애 반경을 줄이고 무권한 relay를 막는다. 인증/인가 정책과 같이 가야 한다.

4단계

Native ingest + adaptive quality

장비 protocol을 살려 송출 부담과 relay Mbps를 줄인다. 임무별 품질 기준이 필요하다.

후순위

Rust TURN 또는 SFU

coturn/MediaMTX 계측에서 실제 CPU 병목이 확인된 뒤 PoC로 검증한다.

참고와 한계

개선 수치는 환경 의존적인 범위 추정치이며 실제 부하 시험으로 고정해야 함

출처와 근거

- WebRTC ICE/STUN/TURN model: MDN WebRTC connectivity, W3C WebRTC stats
- TURN operation: coturn project documentation and common TURN allocation behavior
- GCS-Saker 운영 로그: MediaMTX WHEP, coturn 403, 8189 UDP/TCP 개방 이후 실제 연결 확인
- 내부 PoC 제약: 현재 49160~49200 relay range는 운영 용량 산정 전의 작은 검증 범위

주의

- TURN port 소비량은 BUNDLE, RTCP mux, audio/video m-line, UDP/TCP/TLS 정책, stale allocation 정리에 따라 달라진다.
- N:M viewer 수가 증가하면 TURN보다 먼저 브라우저 decoder, dashboard rendering, viewer network가 병목이 될 수 있으므로 end-to-end 지표를 같이 봐야 한다.
- SFU와 Rust TURN은 장점이 있지만, 현재 단계에서는 direct ICE와 TURN 운영 한계 제거가 우선이다.